

Lecture 22: Typing a Component with TypeScript

INTERVIEW QUESTION

> CORE

How do you use TypeScript inside a `.svelte` component and type its props?

You want type safety in a component. Typed props, typed state. What do you add to the script tag, and how do you declare a prop interface?

The Bug Nobody Could Find

You are building a `<PriceBlock>` component for a shop. It takes a `price` prop, and the parent grid is supposed to hand it a number. One afternoon a teammate wires it up from a different page and, without noticing, passes the price as a string: `price="29.99"` instead of `price={29.99}`.

Nothing errors. The app builds. The page renders. But the little "add insurance" upsell that runs `price + 1` now performs string concatenation instead of addition, so the cart quietly shows `29.991` dollars instead of `30.99`, and a customer screenshots the broken store before anyone on the team notices. The editor never warned you, because plain JavaScript has no idea that `price` was *supposed* to be a number; the mistake sailed straight through the build and detonated in front of a shopper.

That is the failure mode of untyped props: the contract between a parent and a child lives only in the author's head, and a typo or a wrong type becomes a silent runtime bug. The fix is to write that contract down in a form the editor can enforce, so the mistake is caught while you are typing, not after you ship.

The One Line That Turns It On

To bring TypeScript into a Svelte component, you add a single attribute to the script tag: `lang="ts"` .

```
1 <script lang="ts">
2   let name: string = 'world';
3
4   function greet(name: string) {
5     alert(`Hello, ${name}!`);
6   }
7 </script>
```

With that attribute in place, you can use TypeScript's type annotations, its `interface` declarations, and the rest of its type vocabulary throughout the component, and the Svelte tooling plus your editor will check them for you. `svelte-check` does the same job on the command line, so you can run it in CI.

KEY TAKEAWAY

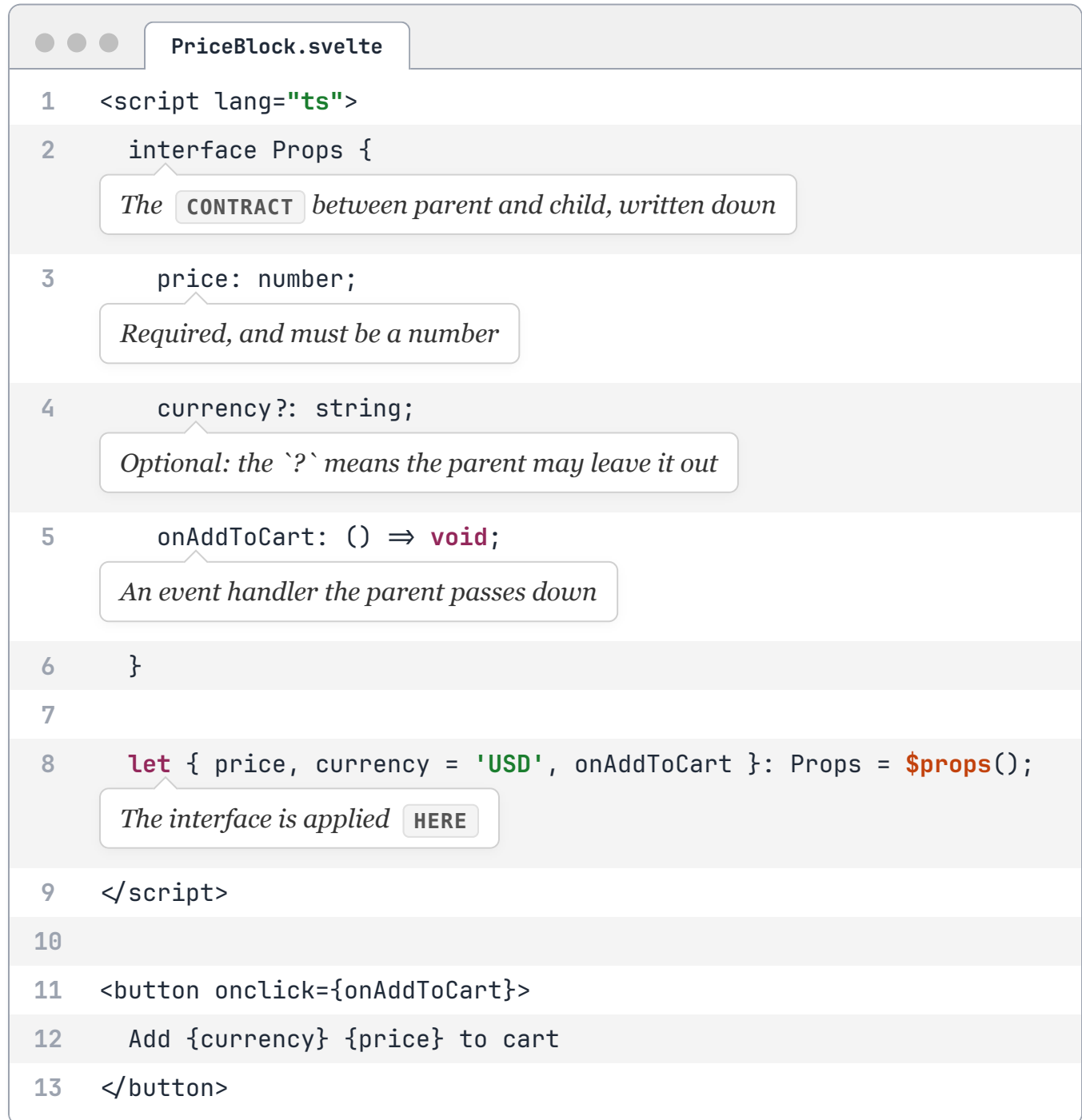
Types are scaffolding, not structure: they guide the build, then vanish without leaving a single byte in the bundle.

Here is the mechanical reason `lang="ts"` needs no extra tooling for the common case. TypeScript has two kinds of features. Type-only features, like annotations and interfaces, describe your code but compile down to nothing: the Svelte

compiler simply strips them out and emits plain JavaScript, so they cost zero bytes and zero runtime. Features that require the TypeScript compiler to *output* real code, like enums, cannot be stripped away, so those need a preprocessor such as `vitePreprocess` to translate them first.

Typing the Props

The direct answer to the interview question is an `interface` describing the component's props, attached to the `$props()` destructuring with a type annotation.



```
1  <script lang="ts">
2    interface Props {
3      price: number;
4      currency?: string;
5      onAddToCart: () => void;
6    }
7
8    let { price, currency = 'USD', onAddToCart }: Props = $props();
9  </script>
10
11  <button onclick={onAddToCart}>
12    Add {currency} {price} to cart
13  </button>
```

PriceBlock.svelte

*The **CONTRACT** between parent and child, written down*

Required, and must be a number

Optional: the `?` means the parent may leave it out

An event handler the parent passes down

*The interface is applied **HERE***

Three things are happening at once. The `interface Props` block declares, prop by prop, what the parent is allowed to

pass: `price` must be a number, `currency` is an optional string with a default, and `onAddToCart` is a function. The `: Props` annotation on the destructuring is what actually binds that contract to the incoming props. And because the editor now knows the shape, the moment a parent writes `price="29.99"` or misspells `prcie`, you get a red squiggle before the file is even saved. The bug from the opening is now impossible to ship.

INTERVIEW TIP

To impress the interviewer: Don't stop at "you add `lang="ts"` and an interface." The answer that signals real depth names *why* it needs no extra tooling: type-only features are erased at compile time, so a typed Svelte component ships the exact same JavaScript as an untyped one. That ties TypeScript directly to Svelte's compiler-first design, and it is the detail that separates a framework user from someone who understands the machinery.

Typing State and Snippets

`$state` is typed like any other variable, though it usually needs no annotation because the initial value already tells TypeScript the type. You spell the type out only when it is wider than the initial value, as in `let note: string | null = $state(null)`, and if you give `$state` no initial value at all, an `as` cast states what it will be once set, which is common inside classes. Snippet props are typed with the `Snippet` interface from `svelte`, where the type argument is a tuple of the snippet's parameters: `row: Snippet<[Product]>` (see Lecture 21 for how snippets work).

The Preprocessor Boundary

Type-only features work the moment you add `lang="ts"`.

The moment you reach for a feature that emits JavaScript, an enum being the classic example, the compiler needs help. In a Vite or SvelteKit project you hand that off with one line of config:



```
1  import { vitePreprocess } from '@sveltejs/vite-plugin-svelte';
2
3  const config = {
4    preprocess: vitePreprocess({ script: true })
5  };
6
7  export default config;
```

*Lets TypeScript **EMIT** code, not just erase it*

In practice, most component code lives entirely in type-only territory, so the plain `lang="ts"` attribute is all you ever touch day to day.

A First Step into Generics

Sometimes the props relate to each other. A list component receives `items` and a `select` callback, and the two must agree on the type of a single item. You express that relationship with the `generics` attribute on the script tag, and the editor then enforces that both sides match for whoever uses the component. This is deeper than the core question, but it is the natural next step once props are typed.

Summary

To use TypeScript in a Svelte component, add `lang="ts"` to the script tag, then declare an `interface Props` describing each prop and attach it to the props destructuring with `let { ... }: Props = $props()`. This works with no extra setup for type-only features, because the Svelte compiler strips types away at build time, leaving the bundle byte-for-byte identical to plain JavaScript; only code-emitting features like enums need a preprocessor such as `vitePreprocess`. State is typed like any variable, snippet props use the `Snippet` interface, and the editor plus `svelte-check` enforce the whole contract so prop mistakes surface at authoring time instead of in production.

	PLAIN JAVASCRIPT (Runtime Discovery)	TYPESCRIPT (Authoring-Time Checks)
Prop mistakes	Silent runtime bug, found by users	Red squiggle the moment you type it
The contract	Lives in the author's head	Written as an <code>interface</code> <code>Props</code>

	PLAIN JAVASCRIPT (Runtime Discovery)	TYPESCRIPT (Authoring-Time Checks)
Setup	None	Add <code>lang="ts"</code> to the script tag
Bundle cost	N/A	Zero: types are erased at build time